Linear algebra: Exercise session 2

Morteza Rahimi

Autumn 2025

---

1. Consider following matrices:

$$A = \begin{bmatrix} 1 & -2 & 1 \\ 2 & 1 & 0 \end{bmatrix},\ B = \begin{bmatrix} 3 & 1 & 1 \\ -2 & 4 & -1 \\ -3 & 1 & 3 \end{bmatrix},\ C = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix},\ D = \begin{bmatrix} -2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & -1 \end{bmatrix}, E = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 3 \end{bmatrix},$$
$$F = \begin{bmatrix} 3 & 3 & 0 \\ 0 & -3 & 1 \\ 0 & 0 & 3 \end{bmatrix},\ G = \begin{bmatrix} 0 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix},\ H = \begin{bmatrix} -4 & 1 & 2 \\ 1 & -3 & -1 \\ 2 & -1 & 7 \end{bmatrix},\ I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},\ E_{11} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix},\ v = \begin{bmatrix} 1 \\ -2 \end{bmatrix}.$$

(a) Determine the type of the following matrices (square, rectangular, diagonal, upper triangular, lower triangular, zero, identity, scalar, elementary, symmetric, (strictly) diagonally dominant).
Answer.

- A is a 2×3 rectangular matrix.
- B is a 3×3 square matrix. It is not symmetric or diagonally dominant.
- C is a 3×2 zero rectangular matrix.
- D is a 3×3 diagonal matrix.
- E is a 3×3 scalar matrix.
- F is a 3×3 upper triangular matrix. It is diagonally dominant. It is not symmetric.
- G is a 3×3 upper triangular matrix. It is not symmetric or diagonally dominant.
- H is a 3×3 symmetric and strictly diagonally matrix.
- I is a 2×2 identity matrix.
- E_{11} is a 2×2 elementary matrix.

– –

(b) Calculate $A^T A$ and $B^T \pm B$.
Answer.

$$A^T A = \begin{bmatrix} 1 & 2 \\ -2 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & -2 & 1 \\ 2 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 5 & 0 & 1 \\ 0 & 5 & -2 \\ 1 & -2 & 1 \end{bmatrix},$$
$$B^T + B = \begin{bmatrix} 3 & -2 & -3 \\ 1 & 4 & 1 \\ 1 & -1 & 3 \end{bmatrix} + \begin{bmatrix} 3 & 1 & 1 \\ -2 & 4 & -1 \\ -3 & 1 & 3 \end{bmatrix} = \begin{bmatrix} 6 & -1 & -2 \\ -1 & 8 & 0 \\ -2 & 0 & 6 \end{bmatrix}, \quad B^T - B = \begin{bmatrix} 0 & -3 & -4 \\ 3 & 0 & 2 \\ 4 & -2 & 0 \end{bmatrix}.$$

– –

(c) Calculate AB and BA, if it is possible.
Answer.

$$AB = \begin{bmatrix} 1 & -2 & 1 \\ 2 & 1 & 0 \end{bmatrix} \begin{bmatrix} 3 & 1 & 1 \\ -2 & 4 & -1 \\ -3 & 1 & 3 \end{bmatrix} = \begin{bmatrix} 4 & -6 & 6 \\ 4 & 6 & 1 \end{bmatrix}, \quad BA \text{ not defined}$$

– –

(d) Calculate $(2H + B - G)^T$, $A^T v$, A^2, F^2, and D^{10}, if it is possible.
Answer.

$$2H + B - G = 2 \begin{bmatrix} -4 & 1 & 2 \\ 1 & -3 & -1 \\ 2 & -1 & 7 \end{bmatrix} + \begin{bmatrix} 3 & 1 & 1 \\ -2 & 4 & -1 \\ -3 & 1 & 3 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} -5 & 3 & 5 \\ -2 & -3 & -3 \\ 2 & -1 & 17 \end{bmatrix},$$

$$(2H + B - G)^T = \begin{bmatrix} -5 & -2 & 2 \\ 3 & -3 & -1 \\ 5 & -3 & 17 \end{bmatrix},$$

$$A^T v = \begin{bmatrix} 1 & 2 \\ -2 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ -2 \end{bmatrix} = \begin{bmatrix} -3 \\ -4 \\ 1 \end{bmatrix},$$

$$A^2 \text{ not defined}, \quad F^2 = F \cdot F = \begin{bmatrix} 3 & 3 & 0 \\ 0 & -3 & 1 \\ 0 & 0 & 3 \end{bmatrix} \begin{bmatrix} 3 & 3 & 0 \\ 0 & -3 & 1 \\ 0 & 0 & 3 \end{bmatrix} = \begin{bmatrix} 9 & 0 & 3 \\ 0 & 9 & 0 \\ 0 & 0 & 9 \end{bmatrix}, \quad D^{10} = \begin{bmatrix} 1024 & 0 & 0 \\ 0 & 1024 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

(e) Calculate $tr\big((-2B^T + 4H)^T\big)$.
Answer.

$$tr\big((-2B^T + 4H)^T\big) = -2\,tr(B) + 4\,tr(H) = -2(3+4+3) + 4(-4-3+7) = -20.$$

---

2. Let A, B be a $n \times n$ matrix. Compute the following expressions:

$$(I+A)^2, \quad (I+A^T)(I+A) \quad (I-A)(I+A), \quad (A+B)^2, \quad (I + A + A^2 + \ldots + A^m)(I - A).$$

Answer.

$$(I+A)^2 = (I+A)(I+A) = I + 2A + A^2,$$
$$(I+A^T)(I+A) = I + A + A^T + A^T A,$$
$$(I-A)(I+A) = I + A - A - AA = I - A^2,$$
$$(A+B)^2 = (A+B)(A+B) = A^2 + AB + BA + B^2,$$
$$(I + A + A^2 + \ldots + A^m)(I - A) = I + A + A^2 + \ldots + A^m - (A + A^2 + \ldots + A^m + A^{m+1}) = I - A^{m+1}.$$

---

3. Write a Python program that asks the user to enter a vector representing the diagonal entries of a matrix. The program should construct and display the diagonal matrix, as well as the zero matrix and the identity matrix of the same size.
Answer.

Listing 1: Constructing diagonal, zero, and identity matrices from user input

```python
import numpy as np

# Step 1: Ask the user to enter the diagonal entries as a space-separated string
diag_entries = list(map(float, input("Enter the diagonal entries separated by spaces: ").split()))

# Step 2: Determine the size of the square matrix
n = len(diag_entries)

# Step 3: Construct the matrices
D = np.diag(diag_entries)    # Diagonal matrix
Z = np.zeros((n, n))         # Zero matrix
I = np.eye(n)                # Identity matrix

# Step 4: Display the matrices
print("\nDiagonal matrix D:")
print(D)

print("\nZero matrix of the same size:")
print(Z)

print("\nIdentity matrix of the same size:")
print(I)
```

---

4. Write a Python program that asks the user to enter a matrix, then displays the matrix along with its size, transpose, and trace. The program should also check whether the matrix is symmetric and whether it is (strictly) diagonally dominant.
Answer.

Listing 2: Matrix properties: size, transpose, trace, symmetry, and diagonal dominance

```python
import numpy as np


# --- Step 1: Get matrix input from the user ---
rows = int(input("Enter the number of rows: "))
cols = int(input("Enter the number of columns: "))


print("Enter the entries row by row, separated by spaces:")
matrix_entries = []
for i in range(rows):
    row = list(map(float, input(f"Row {i+1}: ").split()))
    matrix_entries.append(row)


A = np.array(matrix_entries)


# --- Step 2: Display the matrix ---
print("\nMatrix A:")
print(A)


# --- Step 3: Display the size ---
print("\nSize of A:", A.shape)


# --- Step 4: Display the transpose ---
print("\nTranspose of A:")
print(A.T)


# --- Step 5: Display the trace (only if square) ---
if rows == cols:
    print("\nTrace of A:", np.trace(A))
else:
    print("\nTrace is not defined (matrix is not square).")


# --- Step 6: Check symmetry (only if square) ---
if rows == cols and (A == A.T).all():
    print("\nA is symmetric.")
else:
    print("\nA is not symmetric.")


# --- Step 7: Check diagonal dominance (only if square) ---
def diagonally_dominant(M):
    n = M.shape[0]
    for i in range(n):
        if 2*abs(M[i, i]) < np.sum(np.abs(M[i, :])):
            return False
    return True


# --- Step 7: Check strict diagonal dominance (only if square) ---
def strictly_diagonally_dominant(M):
    n = M.shape[0]
    for i in range(n):
        if 2*abs(M[i, i]) <= np.sum(np.abs(M[i, :])):
            return False
    return True


if rows == cols:
```

```
    if strictly_diagonally_dominant(A):
        print("\nA is strictly diagonally dominant.")
    elif diagonally_dominant(A):
        print("\nA is diagonally dominant, but it is not strictly diagonally dominant.")
    else:
        print("\nA is not strictly diagonally dominant.")
else:
    print("\nDiagonal dominance is not defined (matrix is not square).")
```

5. Write a Python program that asks the user to enter a matrix, a vector, and a scalar. The program should display the matrix, the vector, and the results of the scalar-matrix multiplication and the vector-matrix multiplication.
   Answer.

Listing 3: Matrix and vector operations with scalar

```
import numpy as np

# Step 1: Get matrix input from user
rows = int(input("Enter the number of rows of the matrix: "))
cols = int(input("Enter the number of columns of the matrix: "))

print("Enter the matrix entries row by row, separated by spaces:")
matrix_entries = []
for i in range(rows):
    row = list(map(float, input(f"Row {i+1}: ").split()))
    matrix_entries.append(row)
A = np.array(matrix_entries)

# Step 2: Get vector input from user
print("\nEnter the vector entries separated by spaces:")
v = np.array(list(map(float, input().split())))

# Step 3: Get scalar input from user
s = float(input("\nEnter a scalar value: "))

# Step 4: Display inputs
print("\nMatrix A:")
print(A)

print("\nVector v:")
print(v)

print("\nScalar s:", s)

# Step 5: Scalar-matrix multiplication
print("\nScalar-matrix multiplication (s*A):")
print(s * A)

# Step 6: Matrix-vector multiplication (A*v) if dimensions match
if v.shape[0] == cols:
    print("\nMatrix-vector multiplication (A*v):")
    print(A @ v)
else:
    print("\nMatrix-vector multiplication not possible due to dimension mismatch.")
```

6. Write a Python program that asks the user to enter two matrices of the same size. The program should display their sum and their product (matrix-matrix multiplication).
   Answer.

Listing 4: Matrix addition and multiplication with dimension check

```
import numpy as np

# Step 1: Get matrix dimensions
rows1 = int(input("Enter the number of rows of the first matrix: "))
cols1 = int(input("Enter the number of columns of the first matrix: "))

```

```python
rows2 = int(input("\nEnter the number of rows of the second matrix: "))
cols2 = int(input("Enter the number of columns of the second matrix: "))

# Step 2: Input the first matrix
print("\nEnter the entries of the first matrix row by row, separated by spaces:")
matrix1_entries = []
for i in range(rows1):
    row = list(map(float, input(f"Row {i+1}: ").split()))
    matrix1_entries.append(row)
A = np.array(matrix1_entries)

# Step 3: Input the second matrix
print("\nEnter the entries of the second matrix row by row, separated by spaces:")
matrix2_entries = []
for i in range(rows2):
    row = list(map(float, input(f"Row {i+1}: ").split()))
    matrix2_entries.append(row)
B = np.array(matrix2_entries)

# Step 4: Display matrices
print("\nMatrix A:")
print(A)
print("\nMatrix B:")
print(B)

# Step 5: Check and perform matrix addition
if A.shape == B.shape:
    sum_matrix = A + B
    print("\nSum of A and B (A + B):")
    print(sum_matrix)
else:
    print("\nMatrix addition not possible due to dimension mismatch.")

# Step 6: Check and perform matrix multiplication
if A.shape[1] == B.shape[0]:
    product_matrix = A @ B
    print("\nProduct of A and B (A * B):")
    print(product_matrix)
else:
    print("\nMatrix multiplication not possible due to dimension mismatch.")
```